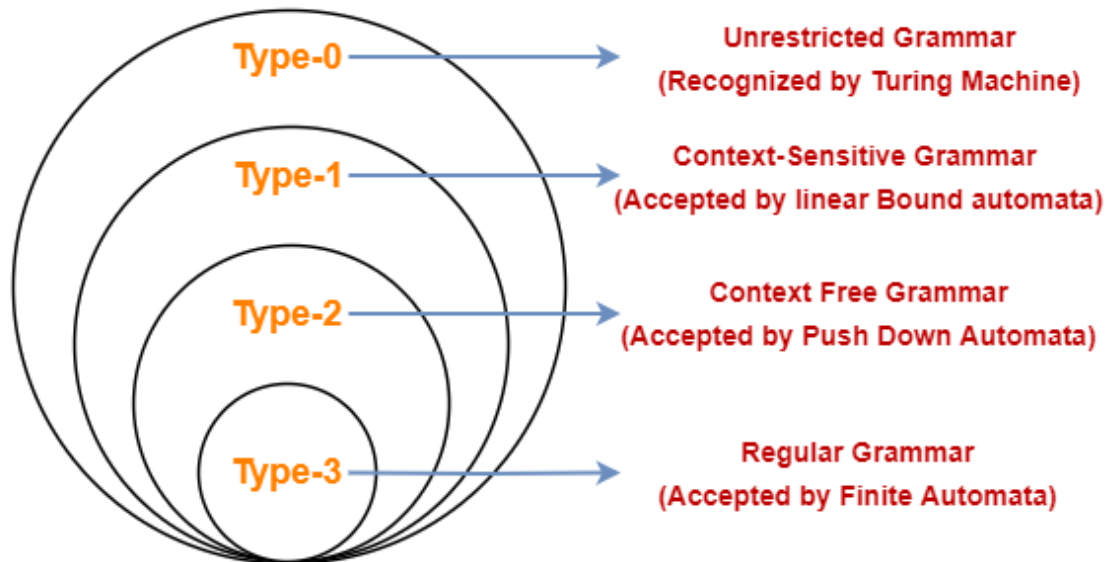
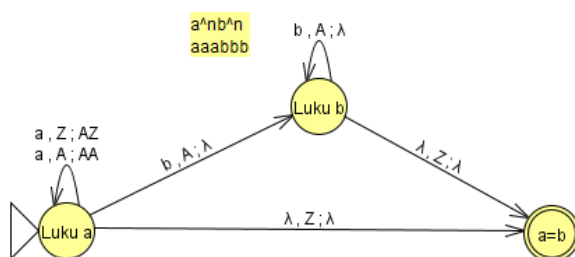


Grammars and parsing



Type 2	Context free grammars	PDA (Push down automaton), Non deterministic push down automaton	$A \rightarrow \alpha$	$L = \{a^n b^n \mid n > 0\}$
--------	-----------------------	--	------------------------	------------------------------



$S \rightarrow aMb$

$M \rightarrow aMb \mid \epsilon$

$S \rightarrow \epsilon$

as a result: $a^n b^n$

NOTE: Regular grammars are context-free, but not (in general) vice versa

Linear grammar: at most one nonterminal on the right-hand side

Right linear grammar

$S \rightarrow aS$

Left linear grammar

$S \rightarrow Sb$

Regular grammar

Is the grammar

$S \rightarrow Sb$

$S \rightarrow aS$

$S \rightarrow \epsilon$

regular? No, it is not, but why ?

A regular grammar is a grammar that is left-linear or right-linear.

Right regular grammar

$A \rightarrow a$

$A \rightarrow aB$

$A \rightarrow \epsilon$

Left regular grammar

$A \rightarrow a$

$A \rightarrow Ba$

$A \rightarrow \epsilon$

Regular grammar:

- All production rules have at most one non-terminal symbol
- A symbol is either always at the end or always at the start of the rule's right-hand side.

As a reminder (lecture 5):

A type 3 regular language is defined as:

The empty language $\emptyset$ is a regular language

$\Sigma = \{a,b,c,d,\dots,z\}$ is a finite set of alphabets, which can be any characters

If A is a regular language, then A^* is also a regular language (Kleene star)

Thus, the empty string $\{\epsilon\}$ is also a regular language

If A and B are regular languages, $A \cup B$ (union) and $A \bullet B$ (concatenation) are regular languages

Other languages over the alphabet Σ are not regular languages

Context-free grammar

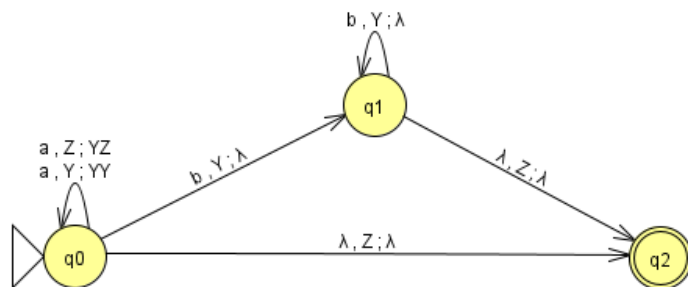
No context on the production rules on the left-hand side of the rule.

$S \rightarrow A$

$A \rightarrow aA \mid aB$

$B \rightarrow bB \mid \lambda$

JFLAP automaton to grammar



$a^n b a^n$

LHS		RHS
S	$\rightarrow$	aAB
S	$\rightarrow$	λ
G	$\rightarrow$	b
A	$\rightarrow$	aAG
A	$\rightarrow$	b
B	$\rightarrow$	λ

Parsing

A parse tree for a grammar G is a tree where

- the root is the start symbol for G
- the interior nodes are the nonterminals of G
- the leaf nodes are the terminal symbols of G.

- the children of a node T (from left to right) correspond to the symbols on the right hand side of some production for T in G.

Having the following rules:

1. $\langle E \rangle \rightarrow \langle D \rangle$
2. $\langle E \rangle \rightarrow (\langle E \rangle)$
3. $\langle E \rangle \rightarrow \langle E \rangle + \langle E \rangle$
4. $\langle E \rangle \rightarrow \langle E \rangle - \langle E \rangle$
5. $\langle E \rangle \rightarrow \langle E \rangle * \langle E \rangle$
6. $\langle E \rangle \rightarrow \langle E \rangle / \langle E \rangle$
7. $\langle D \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots 9$

Let's parse the expression $8 + 3 * 3$ (done on lectures)

Ambiguous Grammars

Ambiguous Grammar = If there are two or more parse trees for the same string of non-terminals.

Another way to parse $8 + 3 * 3$?

